

AI Music Recommendation System 技术论文

主题：基于歌单数据与用户反馈训练混合音乐推荐模型的设计、实现与评估

作者：AI Fair Project Team

版本：适合课堂展示、答辩和项目归档的长篇技术说明稿

摘要

本论文系统性介绍一个本地运行的 AI Music Recommendation System。项目以 Flask 为本地 Web 后端，以 CSV 文件作为轻量级数据存储，以 LightFM 作为主推荐模型，通过 QQ 音乐歌单 metadata 导入、手动歌单输入、免费音乐 metadata API 搜索、Like/Dislike 用户反馈等方式，把真实歌曲、歌单和用户行为转化为机器学习训练数据。系统不使用本地 LLM、不使用 LM Studio、不使用 Ollama、不使用 OpenAI API，也不使用语言模型生成推荐解释。所有推荐理由都由确定性规则根据歌曲特征、用户画像和推荐分数生成。项目的核心价值在于：它不是简单地把几首歌做余弦相似度排序，而是把歌单导入视为 implicit feedback，把 Like/Dislike 视为 explicit feedback，把歌曲转换成 item features，然后训练 LightFM(loss="warp") 混合推荐模型。模型训练完成后，系统会对候选歌曲预测用户偏好分数，再与 content similarity 组合，最终生成 Top-N 推荐和 50 首 Daily Recommendation Mix。

论文重点解释五个问题：第一，系统如何从 QQ 音乐 URL 或歌单 ID 中提取 playlist_id，并通过多个公开 metadata endpoint 尝试获取歌曲列表；第二，系统如何调用 Deezer、MusicBrainz 和 Last.fm 等免费音乐 metadata API 搜索歌曲，并保证 API 失败时 Flask 不崩溃；第三，系统如何把歌曲 metadata 和音频特征转换成适合 LightFM 训练的用户-歌曲交互矩阵与 item feature matrix；第四，LightFM WARP ranking model 如何学习用户偏好并与 content-based baseline 结合；第五，Like/Dislike 反馈如何保存为新的训练数据，并在重新训练后改变推荐结果。本文也讨论 macOS App 打包、数据安全、版权边界、模型局限和未来改进方向。

关键词

音乐推荐系统；LightFM；WARP；混合推荐；隐式反馈；显式反馈；QQ 音乐歌单导入；MusicBrainz；Deezer；Flask；CSV 数据存储；AI Fair

目录

1. 项目背景与问题定义
2. 系统总体架构
3. 数据文件与数据建模
4. QQ 音乐歌单 URL 提取与 metadata 导入
5. 免费音乐 API 搜索模块

6. 特征工程与 item feature token
7. LightFM 混合推荐模型训练
8. Content-Based Baseline 与 Logistic Regression Fallback
9. 推荐排序、重排序与 Daily 50 输出
10. Like/Dislike 反馈学习
11. Flask API 与前端交互流程
12. macOS App 打包与本地运行时设计
13. 安全、版权与隐私边界
14. 测试方案与 AI Fair 展示方式
15. 局限性与未来改进
16. 附录：逐函数技术讲解、答辩问答和扩展说明

1. 项目背景与问题定义

音乐推荐系统的目标是根据用户已有的听歌行为、歌单收藏、搜索与反馈，预测用户可能喜欢的下一批歌曲。商业音乐平台通常拥有大量播放日志、跳过日志、收藏行为、歌单关系、歌曲音频分析结果、歌词、社交关系和上下文信息。本 AI Fair 项目没有这些大规模数据，因此采用一种适合教学展示的方案：让用户导入 QQ 音乐歌单或手动输入歌曲，系统把这些歌曲看作用户当前兴趣的隐式反馈；用户点击 Like 或 Dislike 后，系统把这些反馈看作更强的显式反馈；所有歌曲都存入本地 CSV，所有交互都存入 interactions.csv，最终训练一个 LightFM hybrid recommendation model。

项目最重要的设计选择是把“推荐系统”从普通搜索网页提升为“可训练模型”。如果只根据歌曲音频特征做 cosine similarity，它仍然可以推荐相似歌曲，但机器学习含量比较弱，因为模型没有真正从用户反馈中学习参数。LightFM 的引入让系统可以学习 user embedding、item embedding 以及 item feature embedding，并通过 WARP loss 优化 Top-N 排名。这样，系统不仅知道某首歌在能量、速度、风格上与歌单相似，还能从用户不断增加的 Like/Dislike 行为中调整偏好。

本项目同时受到学校展示环境的限制：需要本地运行、安装简单、界面直观、不能依赖商业付费 API、不能下载音乐音频、不能绕过版权，也不能因为网络 API 不稳定而崩溃。因此系统采用 Flask + pandas + scikit-learn + LightFM + requests + CSV 的轻量架构，并且所有外部接口都使用 timeout 和 try/except。在线 API 只用于搜索公开 metadata，不提供播放、不下载音频、不破解链接。推荐解释不使用语言模型，而是由规则根据 genre、energy、tempo、danceability、valence 等特征生成。

2. 系统总体架构

系统可以理解为五层结构。第一层是前端 UI，位于 templates/index.html、static/css/style.css 和 static/js/main.js，负责歌单导入、手动输入、搜索、训练、推荐、Daily 50、Like/Dislike 和双语展示。第二层是 Flask API 层，位于 app.py，负责路由分发、JSON 输入输出、端口选择和 JSON NaN 安全处理。第三层是数据服务层，位于 services/data_store.py，负责 CSV 文件创建、读取、写入、歌曲标准化、稳定

track_id 生成和交互记录。第四层是外部 metadata 适配层，包括 QQ 音乐导入 services/qq_music_importer.py 和音乐搜索 services/music_search.py。第五层是推荐模型层，包括 feature_engineering.py、training_service.py 和 recommender.py。

从数据流角度看，用户首先通过 QQ 音乐歌单 URL、手动歌单或搜索结果把歌曲加入系统。系统把歌曲保存到 songs.csv，把歌单事件保存到 playlists.csv，把训练行为保存到 interactions.csv。训练服务读取 songs.csv 和 interactions.csv，筛选 has_audio_features=true 的歌曲，构建 LightFM Dataset、interaction matrix、sample weight matrix 和 item feature matrix，然后训练 LightFM(loss="warp")。推荐服务读取训练后的 models/lightfm_model.pkl，对候选歌曲预测偏好分数，同时计算 content similarity，再按 final_score 排序输出推荐。

这种架构的优点是透明。AI Fair 老师可以直接打开 CSV 文件看到训练数据如何产生，也可以打开模型状态看到用户数、歌曲数、交互数和 item feature 数。系统没有隐藏的云端数据库，没有黑盒 LLM，也没有复杂部署要求。对于课堂项目，透明性比复杂性更重要，因为学生需要解释清楚：什么是数据、什么是特征、什么是交互矩阵、什么是训练、什么是预测。

2.1 主要文件职责

文件	技术职责
app.py	Flask 应用入口、API 路由、JSON 安全输出、端口选择。
services/data_store.py	CSV 数据层、字段定义、歌曲标准化、交互和反馈写入。
services/music_search.py	本地 CSV、Deezer、MusicBrainz、Last.fm 的统一搜索适配。
services/qq_music_importer.py	QQ 音乐歌单 ID 提取、多 endpoint metadata 导入、歌曲标准化。
services/feature_engineering.py	音频数值特征分桶、标签抽取、LightFM item feature token 构造。
services/training_service.py	LightFM Dataset、interaction matrix、item feature matrix、模型训练与保存。
services/recommender.py	LightFM 推荐、content-based baseline、LogisticRegression fallback、daily 50 rerank。
mac_app_launcher.py	macOS App 启动器、运行时数据目录、pywebview 原生窗口。

3. 数据文件与数据建模

项目使用本地 CSV 文件作为轻量数据存储。CSV 的优点是直观、容易检查、适合课堂演示、无须安装数据库。数据层通过 ensure_data_files() 保证 data/songs.csv、data/playlists.csv、data/interactions.csv 和 data/feedback.csv 都存在。如果文件不存在，系统自动创建带表头的 CSV；如果 songs.csv 缺少 demo/curated real song 数据，ensure_demo_songs() 会补充真实歌名的展示曲库。

songs.csv 是歌曲主表，它保存歌曲 metadata、音频特征、来源和是否可用于模型训练。playlists.csv 保存歌单导入记录。interactions.csv 是推荐系统最关键的训练表，因为 LightFM 训练需要 user-item interaction。feedback.csv 则保存 Like/Dislike 原始反馈，便于展示 explicit feedback 的来源。feedback.csv 与 interactions.csv 的区别是：feedback.csv 更像用户界面行为日志，而 interactions.csv 是被训练服务消费的机器学习训练表。

3.x songs.csv 字段

字段序号	字段名
1	track_id
2	track_name
3	artist_name
4	album_name
5	genre
6	tags
7	danceability
8	energy
9	valence
10	tempo
11	acousticness
12	instrumentalness
13	speechiness
14	liveness
15	popularity
16	source
17	external_url
18	has_audio_features

3.x playlists.csv 字段

字段序号	字段名
1	playlist_id
2	playlist_name
3	source
4	song_count
5	created_at

3.x interactions.csv 字段

字段序号	字段名
1	interaction_id
2	user_id
3	track_id
4	interaction_type
5	weight
6	timestamp

3.x feedback.csv 字段

字段序号	字段名
1	feedback_id
2	user_id
3	track_id
4	label
5	timestamp

3.5 交互类型与权重

interaction_type	weight	意义
playlist_import	1.0	QQ 音乐歌单导入形成的隐式正反馈。
manual_add	1.0	手动添加歌曲形成的隐式正反馈。
like	2.0	用户明确喜欢，是更强的正反馈。
dislike	-1.0	用户明确不喜欢，训练时不作为 WARP 正样本，重排序时降权。
skip	-0.5	保留给后续扩展的弱负反馈。

4. QQ 音乐歌单 URL 提取与 metadata 导入

QQ 音乐导入模块的目标不是下载音乐，也不是播放音乐，而是导入公开歌单 metadata。metadata 包括歌曲名、歌手、专辑等文本信息。系统会尝试把 QQ 歌单中的歌曲与本地 songs.csv 中的歌曲匹配；如果匹配成功，就复用本地已有音频特征；如果匹配失败，就根据在线 metadata 和规则估算特征，并标记来源为 qq。导入成功的歌曲会写入 songs.csv，并为 demo_user 创建 playlist_import interaction。

QQ 音乐链接格式并不统一。桌面端、移动端、分享页和旧版页面可能使用不同 URL 结构。因此 extract_qq_playlist_id(input_text) 先使用 urlparse 和 parse_qs 解析 query 参数，再使用多个正则表达式匹配 playlist/id/disstid/tid。这样的多策略解析比只支持一种 URL 更稳健，也更适合真实展示环境。

4.1 支持的 QQ 输入格式

序号	格式
1	纯数字歌单 ID，例如 123456789。
2	https://y.qq.com/n/ryqq/playlist/xxxxx 形式。
3	https://i.y.qq.com/n2/m/share/details/taoge.html?id=xxxxx 形式。
4	URL query 中的 id、disstid、tid 参数。
5	短文本文中的 disstid=xxxxx 或 playlist/xxxxx。

4.2 QQ endpoint fallback 设计

import_qq_playlist() 在识别到 playlist_id 后，不把希望寄托在单个接口上，而是依次尝试 _fetch_with_qzone_endpoint、_fetch_with_v8_endpoint 和 _fetch_with_musicu_endpoint。三个 endpoint 都只请求公开 metadata，并设置 User-Agent、Referer 和 timeout=8。如果某个 endpoint 失败，函数捕获异常，记录 failure_reason，然后继续尝试下一个 endpoint。只有所有 endpoint 都失败时，系统才返回友好错误：QQ Music import failed. Please try manual playlist input or song search.

这种 fallback 设计非常重要，因为非官方公开接口经常受到地区、权限、私有歌单、接口改版和频率限制影响。如果没有 try/except，Flask 后端可能直接抛出异常，前端会显示 500 错误。当前实现把失败作为正常业务情况处理：它返回 success=false、playlist_id、tried_endpoints 和 failure_reason，前端可以告诉用户改用手动输入或搜索歌曲。

4.3 normalize_qq_song 的作用

QQ 返回的 raw_song 结构并不总是统一。有些接口返回 songInfo，有些直接返回 songname、title、name、singer、album 等字段。normalize_qq_song() 首先从不同字段中提取 title、artist 和 album；然后调用 match_local_song(title, artist) 尝试匹配本地曲库。如果找到本地歌曲，就保留本地音频特征并把 source 改为 qq；如果找不到，就调用 build_estimated_song_row() 生成一条估算特征的歌曲行。最后 save_imported_playlist() 把歌曲 upsert 到 songs.csv，并且只为 has_audio_features=true 的歌曲创建 playlist_import interaction。

这个设计体现了推荐系统中的一个现实问题：很多在线 metadata API 不提供 Spotify-style audio features。如果完全拒绝这些歌曲，QQ 导入会显得很弱；如果无条件把无特征歌曲加入训练，模型又无法训练。因此项目采用折中方案：优先本地匹配，匹配失败则估算风格特征，并在 tags 中标记 estimated-features, online-metadata。对于课堂展示，这能解释“真实工业系统也需要特征补全和冷启动处理”。

5. 免费音乐 API 搜索模块

搜索模块的入口是 search_music(query, limit)。它先搜索本地 CSV，如果本地结果已经足够，就直接返回；如果不足，再依次调用 Deezer、MusicBrainz 和 Last.fm。这个顺序是有工程理由的：本地 CSV 最可靠且拥有训练特征；Deezer 对主流歌曲 metadata 返回较干净；MusicBrainz 是开放音乐知识库但结果更宽泛；Last.fm 需要 API key，因此作为可选补充。所有网络请求都设置 timeout=8，并用 try/except 捕获异常，确保 API 失败不会让 Flask 崩溃。

Provider	实现方式	项目价值
Local CSV	无网络依赖，优先返回已有音频特征的歌曲。	速度快、稳定、适合训练。
Deezer	调用 https://api.deezer.com/search，timeout=8。	返回主流歌曲 metadata、歌手、专辑、外部链接和 rank。
MusicBrainz	调用 recording endpoint，设置 User-Agent，并做 1 秒限速。	开放音乐知识库，适合补充 metadata。
Last.fm	需要 LASTFM_API_KEY，缺少 key 时自动跳过。	可选补充来源，不影响系统运行。

5.1 统一返回格式

不同 API 的字段名称不同。Deezer 使用 title、artist.name、album.title；MusicBrainz 使用 recordings、artist-credit、releases；Last.fm 使用 name、artist、url。为了让前端和训练模块不关心 provider 差异，_format_result() 会把所有搜索结果统一成 SONG_COLUMNS 字段结构，包括 track_id、track_name、artist_name、album_name、source、external_url 和 has_audio_features。

track_id 由 stable_track_id(track_name, artist_name, source) 生成。它把 track name、artist name 和 source 拼接后做 SHA-1 hash，再加上 slug 化的歌名，既可读又稳定。这样同一首从同一 source 来的歌每次生成相同 ID，减少重复写入。对于在线结果，build_estimated_song_row() 会先尝试 match_local_song()，若本地已有歌曲就复用真实或 curated 特征；否则用 artist/title hints 推断 genre，再从 GENRE_FEATURES 中取一组合理音频特征。

5.2 MusicBrainz rate limit 与 User-Agent

MusicBrainz 对公共 API 的使用有明确礼貌要求：客户端应设置可识别的 User-Agent，并避免高频请求。因此 search_musicbrainz() 使用全局变量 MUSICBRAINZ_LAST_REQUEST 记录上一次请求时间，如果距离上次请求不足 1 秒，就 sleep 到 1 秒后再发请求。User-Agent 设置为 AI-Fair-Music-Recommendation-System/1.0 (educational demo)。这不仅是技术细节，也是项目伦理的一部分：免费开放 API 不是无限资源，教育项目应该尊重服务规则。

6. 特征工程与 item feature token

LightFM 的 hybrid 推荐能力来自两个输入：user-item interactions 和 item features。interactions 告诉模型“哪个用户与哪首歌发生过正向行为”；item features 告诉模型“这首歌有什么属性”。如果只把歌曲 ID 放进 LightFM，模型只能学习某个用户与某个 item 的关系，对新歌和稀疏数据帮助有限。加入 genre、artist、tags、source 和音频特征分桶后，模型可以学习“用户喜欢 high energy”“用户喜欢 Pop”“用户喜欢 The Weeknd 风格”等更可泛化的偏好。

feature_engineering.py 的核心函数是 build_item_feature_tokens(song_row)。它会生成三类 token：第一类是文本 metadata token，如 genre:pop、artist:the_weeknd、source:qq；第二类是 tag token，如 tag:dance、tag:happy、tag:rock；第三类是音频数值分桶 token，如 energy:high、tempo:fast、valence:sad、danceability:medium。build_item_features_matrix(songs_df, dataset) 再把每首歌的 token 列表交给 LightFM Dataset.build_item_features()，生成稀疏 item feature matrix。

特征	分桶规则	含义
energy	>= 0.7 high；<= 0.4 low；否则 medium	描述歌曲强度、冲击感和兴奋程度。
valence	>= 0.65 positive；<= 0.4 sad；否则 neutral	描述情绪正向程度。
tempo	>= 125 fast；<= 90 slow；否则 medium	描述 BPM 速度区间。
danceability	>= 0.7 high；<= 0.4 low；否则 medium	描述律动和舞曲倾向。
acousticness	>= 0.6 high	描述原声和非电子化程度。
speechiness	>= 0.25 high	描述说唱、人声讲话和 rap 倾向。
popularity	>= 75 high	描述大众流行程度。

6.1 为什么不直接把全部数值喂给 LightFM

LightFM 的 item_features 输入更适合稀疏 token 特征。虽然歌曲有 danceability、energy、valence、tempo 等连续数值，但直接作为连续值并不是当前代码使用的接口风格。将连续特征离散化为 high/medium/low token 有三个优点。第一，可解释性强，老师可以直接理解 energy:high 代表高能量歌曲。第二，适合稀疏矩阵，LightFM 可以为每个 token 学习 embedding。第三，便于生成规则解释，例如“This song shares high energy and fast tempo with your playlist。”对于 AI Fair 展示，可解释性与模型训练同样重要。

7. LightFM 混合推荐模型训练

训练模块位于 services/training_service.py，核心类是 RecommendationTrainer。训练流程由 train_lightfm_model() 触发。首先 load_data() 读取 songs.csv 和 interactions.csv，并筛选 has_audio_features=true 的歌曲作为 feature_songs_df。然后 get_training_status() 检查训练条件：至少 1 个 user、至少 5 条正向 interactions、至少 10 首可训练歌曲。如果条件不足，系统返回“Not enough data to train LightFM model. Using content-based baseline.”，前端不会崩溃，而是继续使用 baseline 推荐。

当 LightFM 可用且数据足够时，build_dataset() 会收集 users、items 和 item feature tokens，调用 Dataset.fit(users=users, items=items, item_features=features)。build_interactions() 会过滤出 weight>0 的交互，这是因为 LightFM WARP 优化的是正样本排名，不适合直接把负权重 dislike 放入正样本矩阵。dislike 仍然保存在 interactions.csv 中，但在 rerank 阶段进行降权。build_item_features() 则构建 item feature matrix。最后模型以 LightFM(loss="warp", random_state=42) 初始化，训练 30 epochs，num_threads=1。

7.1 WARP loss 的教学解释

WARP 是 Weighted Approximate-Rank Pairwise 的缩写，常用于 Top-N recommendation。普通分类模型可能关注“用户是否喜欢某首歌”的二分类准确率，但推荐系统更关心“用户最可能喜欢的歌曲能否排在前面”。WARP 的思想是：对于用户已经喜欢或添加过的正样本，模型希望它的预测分数高于未观察到的负样本。当正样本没有排得足够靠前时，loss 会推动模型更新 user embedding、item embedding 和 feature embedding。

在本项目中，playlist_import、manual_add 和 like 都会形成正向训练信号。like 的权重是 2.0，因此比普通导入歌曲更强。LightFM 的模型可以写成直观形式：预测分数约等于用户向量与歌曲向量的相似度，再加上 item feature embedding 对歌曲表示的贡献。虽然课堂展示不需要推导全部梯度，但可以解释：模型通过反复比较用户喜欢的歌和候选歌，学习哪些 genre、artist、energy、tempo、mood 更符合用户口味。

7.2 模型保存格式

训练完成后 save_model() 使用 pickle 把模型和映射保存到 models/lightfm_model.pkl。保存内容包括 model、dataset、user_id_map、item_id_map、item_feature_names 和 trained_at。user_id_map 和 item_id_map 非常重要，因为 LightFM 内部使用整数索引，而 CSV 使用字符串 ID。推荐时，recommend_lightfm() 必须把 demo_user 映射到 internal_user_id，把 track_id 映射到 internal_item_id，才能调用 model.predict()。

模型状态 API 会返回 `model_type`、`main_model`、`lightfm_available`、`is_trained`、`num_users`、`num_items`、`num_interactions`、`num_item_features`、`last_trained_at` 和 `message`。这些字段不仅供前端显示，也帮助 AI Fair 展示：观众可以看到“训练模型”不是按钮装饰，而是真的统计了训练数据并保存了模型。

8. Content-Based Baseline 与 Logistic Regression Fallback

推荐系统必须处理冷启动：当用户还没有足够交互，或者 LightFM 安装失败，或者模型文件不存在时，系统仍然要能推荐。`recommend_content_based()` 使用 `AUDIO_FEATURE_COLUMNS` 计算内容相似度。它读取候选歌曲，排除用户已有交互歌曲，调用 `_content_score_map(user_id)` 生成每首候选歌与用户画像的相似度。用户画像是用户正向交互歌曲的标准化音频特征平均向量；如果用户没有正向交互，则用流行度最高的若干歌曲构造默认画像。

标准化过程是 $z=(x-\text{mean})/\text{std}$ 。标准化后，系统计算候选歌曲向量与用户 profile 向量的 cosine similarity，并把范围从 $[-1,1]$ 映射到 $[0,1]$ 。这样 `content_score` 越高，代表歌曲音频特征越接近用户已有歌单。这个 baseline 不需要训练 LightFM，但仍有清楚的数学基础。

如果用户已经有 Like 和 Dislike 两类反馈，`_logistic_feedback_scores()` 会训练 LogisticRegression。训练样本来自用户反馈过的歌曲，特征是 `AUDIO_FEATURE_COLUMNS`，标签是 `like=1`、`dislike=0`。`fallback_final_score = 0.6 * content_score + 0.4 * logistic_probability`。这样即使 LightFM 不可用，系统仍然可以展示一个“有监督反馈模型”如何影响推荐。

9. 推荐排序、重排序与 Daily 50 输出

推荐入口 `recommend_for_user(user_id, top_n)` 优先加载 LightFM 模型。如果模型存在，系统先调用 `recommend_lightfm()` 获取比最终 `top_n` 更大的候选池，例如 `top_n*4` 或至少 30 条。然后调用 `_content_score_map()` 计算内容相似度，再由 `rerank_recommendations()` 组合模型分和内容分。核心公式是 $\text{final_score} = 0.7 * \text{model_score} + 0.3 * \text{content_score}$ 。`model_score` 来自训练后的 LightFM，是模型学到的用户偏好；`content_score` 来自音频特征相似度，是可解释的内容距离；`final_score` 结合两者，避免模型过度依赖稀疏交互，也避免 baseline 只推荐声音相似但用户未必喜欢的歌曲。

Daily 50 功能由 `daily_recommendations(user_id, top_n=50)` 实现。它会请求更大的 LightFM 候选池，例如 `top_n*5` 或 160 条，再重排序并调用 `_diversify()`。多样性函数会限制同一 artist 和同一 genre 在早期 pass 中出现过多，避免最终 50 首全部来自同一个风格或同一个歌手。它还用 `seen_titles` 去重，防止不同 source 返回相同歌名和歌手造成重复。

系统还特别过滤了早期 Demo 占位歌。`_candidate_songs()` 会排除 `track_name` 匹配 Demo ... Track 数字和 `artist_name` 匹配 AI Fair Artist 数字的行，保留真实歌名或 curated real song library。这是为了让 AI Fair 展示看起来像真实音乐推荐 App，而不是随机 demo 数据。

10. Like/Dislike 反馈学习

用户点击 Like 或 Dislike 时，前端调用 `POST /api/feedback`，提交 `track_id` 和 `label`。后端 `add_feedback()` 先写入 `feedback.csv`，`label=1` 表示 like，`label=0` 表示 dislike。然后 `add_feedback()` 会同步调用 `add_interaction()`，把 like 写为 `interaction_type=like`、`weight=2.0`，把 dislike 写为 `interaction_type=dislike`、`weight=-1.0`。API 最后调用 `trainer().train_lightfm_model()` 自动重新训练。

这里需要特别解释 dislike 的处理。LightFM WARP 在本项目中只使用 `weight>0` 的交互构建训练矩阵，因此 dislike 不直接作为负权重喂给 LightFM。原因是 WARP 的训练接口主要围绕正样本排名优化，负权重可能造成语义不清或训练不稳定。系统没有丢掉 dislike，而是在 `rerank_recommendations()` 中把 disliked track 的 `final_score` 减去 0.35，并且 `_excluded_track_ids()` 会排除用户已经交互过的歌曲。这样 dislike 既影响推荐，又不会破坏 LightFM 的正样本学习假设。

11. Flask API 与前端交互流程

Flask API 的设计目标是稳定、清晰、适合前端直接调用。所有返回都通过 `safe_jsonify()`，它会递归调用 `json_safe()`，把 NaN、inf、numpy integer、numpy floating 转换成 JSON 安全值。这一设计来自实际 debugging：如果 pandas/numpy 的 NaN 直接进入 jsonify 或前端 `JSON.parse`，浏览器会报 `Unexpected token 'N'`，因为 NaN 不是标准 JSON。`json_safe()` 把这些值转换为 None 或空字符串，保证 50 首推荐也不会因为某个字段 NaN 而导致整个响应不可解析。

API	功能说明
GET /	返回首页，确保 CSV 数据文件存在。
GET /api/health	返回系统健康状态，供前端启动时确认后端可用。
GET /api/search?q=	搜索歌曲，先查本地 CSV，再查在线免费 metadata API。
POST /api/import/qq	导入 QQ 音乐歌单 metadata，不下载音频。
POST /api/playlist/manual	接收手动输入歌单，匹配或估算音频特征，并写入训练交互。
POST /api/training/add	把单首搜索结果加入训练数据。
POST /api/train	训练 LightFM WARP hybrid recommender。
GET /api/model-status	返回模型是否训练、用户数、歌曲数、交互数、item feature 数。
GET /api/recommendations	返回 Top-N 推荐和用户口味画像。
GET /api/daily-recommendations	返回最多 50 首最终每日推荐，并做轻量多样性处理。
POST /api/feedback	保存 Like/Dislike，写入 feedback.csv 和 interactions.csv，然后触发重新训练。
GET /api/taste-profile	返回用户口味画像。

12. macOS App 打包与本地运行时设计

项目不仅可以 `python app.py` 运行，也被打包成 macOS App。`mac_app_launcher.py` 是 App 启动器。它首先调用 `prepare_runtime_data()`，把打包内置的 `data/` 和 `models/` 复制到用户目录 `~/Library/Application Support/AI Music Recommendation System`。这样做的原因是 .app bundle 内部不适合作为长期可写数据库；用户安装到 Applications 后，应用需要一个稳定可写目录保存新的 `songs.csv`、`feedback.csv`、`interactions.csv` 和 `lightfm_model.pkl`。

启动器设置 `AI_MUSIC_DATA_DIR` 和 `AI_MUSIC_MODELS_DIR` 环境变量，让 `services/data_store.py` 使用 Application Support 下的运行时数据目录。然后它导入 Flask app 和 `choose_port`，选择 5000、5001、5002、5003、8000、8080 中可用端口，后台线程启动 Flask server，再用 `pywebview` 创建原生 macOS 窗口加载本地 URL。用户看到的是一个 Mac 应用窗口，而不是手动打开浏览器。PyInstaller spec 把 `templates`、`static`、`data`、`models`、`docs`、`README`、`license`、`LightFM`、`webview`、`PyObjC` 相关模块都打进 .app。

这种方案兼顾了工程可行性和用户体验。Flask 后端仍然保持原结构，不需要重写成 Electron 或 Swift；pywebview 使用系统 WebKit 显示页面，体积比完整 Chromium 方案小；Application Support 目录保证用户反馈和训练结果可持久保存；打包内置数据和模型保证新电脑第一次打开就能使用推荐功能。

13. 安全、版权与隐私边界

项目明确不使用 local LLM、LM Studio、Ollama、OpenAI API，也不向任何语言模型发送歌曲信息。推荐解释由规则生成，因此可复现、可解释、不会产生幻觉。QQ 音乐导入只导入公开歌单 metadata，不下载音乐、不破解播放链接、不绕过版权限制、不获取付费音频。Deezer、MusicBrainz 和 Last.fm 搜索也只用于 metadata 和外部 URL 展示。

数据安全方面，项目只在当前项目目录或 macOS Application Support 的应用专属目录写入 CSV 和模型文件。它不清空文件夹，不删除用户数据，不扫描用户全盘音乐库，也不上传用户歌单到自己的服务器。网络请求只发生在用户搜索或导入时，且每个 requests 调用都有 timeout=8 和 try/except。Last.fm API key 放在 .env 中，缺少时自动跳过，不会阻止系统运行。

14. 测试方案与 AI Fair 展示方式

展示时可以按以下顺序操作。第一，打开 App 或运行 python app.py，展示首页、双语切换和模型状态。第二，搜索 Shape of You，说明系统先查本地 CSV，不足再查在线 API。第三，手动添加一个小歌单，例如 Shape of You - Ed Sheeran、Blinding Lights - The Weeknd、Believer - Imagine Dragons。第四，点击 Train Model，展示 LightFM WARP 模型状态中的用户数、歌曲数、交互数和 item feature 数。第五，点击 Get Recommendations，解释 model score、content score 和 final score。第六，点击 Generate Final 50，展示 Daily Recommendation Mix。第七，对几首歌点击 Like/Dislike，再重新获取推荐，说明反馈已经写入 CSV 并触发重新训练。

答辩重点应该放在“从数据到模型”的完整链路：歌单导入不是最终目的，导入后的歌曲变成 user-item interactions；歌曲 metadata 不是最终目的，它们被转换成 genre、artist、tag、energy bucket、tempo bucket 等 item features；Like/Dislike 不是简单按钮，它们会写入 feedback.csv 和 interactions.csv；训练不是模拟文字，而是真正调用 LightFM(loss="warp") 训练模型并保存 pkl。这样老师能清楚看到项目符合 train a recommendation model 的要求。

15. 局限性与未来改进

当前项目仍有局限。第一，很多在线 metadata API 不提供真实音频特征，因此系统使用基于 genre 和 artist/title hints 的估算特征。估算特征适合展示，但不等同于专业音频分析。未来可以接入合法的音频特征来源或使用本地音频分析工具对用户拥有版权的音频文件提取特征。第二，数据规模较小，LightFM 的效果受交互数量限制。未来可以让多个 demo users 参与，构建更丰富的 user-item matrix。第三，QQ 音乐公开接口不稳定，未来可以提供更完善的手动导入模板，例如粘贴“歌名 - 歌手”列表或 CSV 导入。第四，模型评估目前偏展示导向，未来可以加入 train/test split、precision@k、recall@k、AUC 和用户反馈变化曲线。

从工程角度看，CSV 简单透明，但并发写入能力有限。未来如果要变成真实产品，可以迁移到 SQLite 或 PostgreSQL。前端也可以加入推荐历史、导入历史、模型训练曲线和特征贡献可视化。Daily 50 可以加入

更复杂的 diversity-aware reranking，例如最大边际相关性 MMR，或者对同一歌手、同一年代、同一 BPM 区间设置更细的约束。

16. 附录 A：逐函数技术讲解

resource_path

解决 PyInstaller 打包后模板和静态资源路径变化问题。普通运行时使用项目根目录，frozen 运行时使用 sys._MEIPASS。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

safe_jsonify/json_safe

递归清理 NaN、inf 和 numpy 类型，避免前端 JSON.parse 报错。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

choose_port

按 5000、5001、5002、5003、8000、8080 顺序选择可用端口，减少端口冲突。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向

老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

stable_track_id

基于歌名、歌手和 source 生成稳定 ID，兼顾可读性和去重。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

upsert_songs

向 songs.csv 写入新歌曲，但不重复插入已有 track_id。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

match_local_song

先精确匹配歌名+歌手，再按歌名匹配，用于补全 QQ 或在线结果的本地音频特征。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

extract_qq_playlist_id

用 urlparse、parse_qs 和多个正则表达式识别不同 QQ 分享链接中的歌单 ID。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

import_qq_playlist

多 endpoint fallback 获取 metadata，失败时返回友好错误而不是抛出 Flask 500。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

search_music

本地优先，在线补充，provider 之间去重，统一输出结构。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

build_estimated_song_row

在线 metadata 无音频特征时，根据本地匹配或规则估算 genre/audio features。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

build_item_feature_tokens

把歌曲转为 LightFM 可学习的稀疏 token。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

train_lightfm_model

检查训练条件、构建矩阵、训练 LightFM WARP、保存模型。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

recommend_lightfm

加载模型映射，预测候选歌曲分数，并返回 normalized model score。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

_content_score_map

标准化音频特征，计算用户 profile 与候选歌曲的 cosine similarity。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

_logistic_feedback_scores

在 fallback 模式下用 Like/Dislike 训练 LogisticRegression。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

rerank_recommendations

组合 LightFM score 与 content score，并处理 dislike 降权。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

_diversify

生成 Daily 50 时限制同歌手和同风格过度集中。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

prepare_runtime_data

macOS App 首次启动时把内置 data/models 复制到 Application Support。

技术细节 1：从数据流角度看，该函数的输入并不是孤立的参数，而是整个推荐链路中的一个节点。它的输出会继续影响后续的 CSV 写入、矩阵构建、模型训练或前端展示。因此设计时必须保证返回值稳定、字段命名统一、异常处理清楚。

技术细节 2：从课堂讲解角度看，可以把这个函数解释为系统中的一个可观察步骤。它不是黑盒，而是把用户行为或外部 metadata 转换成下一步可以消费的数据结构。这样的拆分让项目更容易 debug，也更容易向老师说明每一步为什么属于机器学习流程。

技术细节 3：从健壮性角度看，该函数周围通常配合 try/except、默认值、去重或条件判断。音乐 metadata 非常不干净，歌名、歌手、专辑、外部链接和音频特征都可能缺失。如果不做防御式编程，一个空字段就可能导致训练失败或前端渲染失败。

17. 附录 B：答辩问答稿

Q：为什么说这个项目是 Machine Learning，而不是普通搜索？

A：因为系统会把用户与歌曲之间的交互构造成 user-item interaction matrix，并用 LightFM WARP 训练一个 ranking model。搜索只返回匹配关键词的歌曲，而推荐模型会根据用户歌单和反馈预测未听歌曲的偏好分数。

Q：为什么选择 LightFM？

A：LightFM 支持 hybrid recommendation，可以同时使用 interaction data 和 item features。它适合小型项目、Top-N 推荐、隐式反馈和显式反馈混合场景。

Q：QQ 音乐导入是否下载歌曲？

A：不会。系统只请求公开 metadata，例如歌名、歌手和专辑，不下载音频、不破解播放链接、不绕过版权。

Q：如果 QQ API 失败怎么办？

A：系统返回友好错误，并建议使用手动歌单或搜索歌曲。失败不会导致 Flask 崩溃。

Q：在线 API 搜到的歌曲没有真实音频特征怎么办？

A：系统先尝试匹配本地曲库，匹配不到时根据 genre 和 artist/title hints 估算展示用特征，并在 tags 中标记 estimated-features。

Q：Like 和 Dislike 怎样影响模型？

A：Like 会以更高权重写入 interactions.csv，重新训练时成为强正样本。Dislike 会保存为负反馈，并在重排序阶段降低对应歌曲分数。

Q：推荐理由是不是 LLM 生成的？

A：不是。所有解释都是 deterministic rule，根据 genre、energy、tempo、danceability、valence 和用户画像生成。

Q：为什么需要 content-based baseline？

A：当训练数据不足或 LightFM 不可用时，系统仍然需要推荐。baseline 用音频特征相似度解决冷启动。

Q：Daily 50 如何避免全是同一种歌？

A：系统先取较大候选池，再使用 `_diversify` 限制同歌手和同风格过度集中，并去重歌名歌手组合。

Q：打包成 macOS App 后数据存在哪里？

A：首次启动时内置 `data/models` 会复制到 `~/Library/Application Support/AI Music Recommendation System`，之后用户反馈和重新训练模型都写入这里。

18. 附录 C：扩展技术讨论

18.1 数据稀疏性 扩展说明

推荐系统最常见的问题是 user-item matrix 极其稀疏。本项目虽然只有 `demo_user`，但仍然通过 item features 缓解稀疏性。LightFM 不只学习某首歌的 ID，也学习 genre、artist、tag 和音频分桶 token，因此即使某首候选歌从未被用户交互过，只要它拥有相似 item features，模型仍然可以给出合理分数。

在本项目中，数据稀疏性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.2 冷启动 扩展说明

冷启动包括用户冷启动和物品冷启动。新用户没有交互时，系统使用热门歌曲或默认内容画像；新歌曲没有交互时，只要有 item features，LightFM 和 content-based baseline 都能参与排序。

在本项目中，冷启动并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.3 特征估算 扩展说明

在线 API 通常提供 metadata 而不是音频分析结果。项目使用 genre hints 和 artist/title hints 估算特征，是教学项目中的可解释折中。估算结果不能声称为真实音频分析，但可以支撑训练流程展示。

在本项目中，特征估算并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.4 可解释性 扩展说明

推荐解释不依赖语言模型，而是由 _matched_features 规则生成。这样解释可以追溯到具体字段，例如 genre、tempo 和 energy。对于课堂项目，确定性解释比生成式解释更可靠。

在本项目中，可解释性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.5 鲁棒性 扩展说明

所有网络请求都 timeout=8 并捕获异常，所有 JSON 返回都经过 NaN 清洗，所有 CSV 都由 ensure_data_files 自动创建。这些工程细节保证展示时即使网络失败也不会整站崩溃。

在本项目中，鲁棒性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.6 模型评估 扩展说明

项目当前重展示流程，未来可以加入 precision@k、recall@k、AUC、coverage、diversity 等指标。LightFM 自带 evaluation 工具，可以用于更正式的实验报告。

在本项目中，模型评估并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.7 版权边界 扩展说明

项目只处理 metadata，不下载或播放音乐。它尊重音乐平台版权限制，适合作为教育项目展示推荐系统，而不是音乐播放或下载工具。

在本项目中，版权边界并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.8 部署形态 扩展说明

python app.py 适合开发，PyInstaller + pywebview 适合交给同学或老师使用。两者复用同一套 Flask 后端和前端代码，降低维护成本。

在本项目中，部署形态并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.9 数据稀疏性 扩展说明

推荐系统最常见的问题是 user-item matrix 极其稀疏。本项目虽然只有 demo_user，但仍然通过 item features 缓解稀疏性。LightFM 不只学习某首歌的 ID，也学习 genre、artist、tag 和音频分桶 token，因此

即使某首候选歌从未被用户交互过，只要它拥有相似 item features，模型仍然可以给出合理分数。

在本项目中，数据稀疏性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.10 冷启动 扩展说明

冷启动包括用户冷启动和物品冷启动。新用户没有交互时，系统使用热门歌曲或默认内容画像；新歌曲没有交互时，只要有 item features，LightFM 和 content-based baseline 都能参与排序。

在本项目中，冷启动并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.11 特征估算 扩展说明

在线 API 通常提供 metadata 而不是音频分析结果。项目使用 genre hints 和 artist/title hints 估算特征，是教学项目中的可解释折中。估算结果不能声称为真实音频分析，但可以支撑训练流程展示。

在本项目中，特征估算并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.12 可解释性 扩展说明

推荐解释不依赖语言模型，而是由 `_matched_features` 规则生成。这样解释可以追溯到具体字段，例如 `genre`、`tempo` 和 `energy`。对于课堂项目，确定性解释比生成式解释更可靠。

在本项目中，可解释性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 `interaction matrix`；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 `query`，处理步骤是本地搜索和在线 `provider fallback`，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 `Dataset`、`interactions`、`item features` 和 `fit`，输出是 `pkl` 模型，设计原因是 `hybrid recommendation`，失败处理是 `fallback baseline`。

18.13 鲁棒性 扩展说明

所有网络请求都 `timeout=8` 并捕获异常，所有 JSON 返回都经过 NaN 清洗，所有 CSV 都由 `ensure_data_files` 自动创建。这些工程细节保证展示时即使网络失败也不会整站崩溃。

在本项目中，鲁棒性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 `interaction matrix`；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 `query`，处理步骤是本地搜索和在线 `provider fallback`，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 `Dataset`、`interactions`、`item features` 和 `fit`，输出是 `pkl` 模型，设计原因是 `hybrid recommendation`，失败处理是 `fallback baseline`。

18.14 模型评估 扩展说明

项目当前重展示流程，未来可以加入 `precision@k`、`recall@k`、`AUC`、`coverage`、`diversity` 等指标。LightFM 自带 `evaluation` 工具，可以用于更正式的实验报告。

在本项目中，模型评估并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 `interaction matrix`；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 `query`，处理步骤是本地搜索和在线 `provider fallback`，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 `Dataset`、`interactions`、`item features` 和 `fit`，输出是 `pkl` 模型，设计原因是 `hybrid recommendation`，失败处理是 `fallback baseline`。

18.15 版权边界 扩展说明

项目只处理 metadata，不下载或播放音乐。它尊重音乐平台版权限制，适合作为教育项目展示推荐系统，而不是音乐播放或下载工具。

在本项目中，版权边界并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.16 部署形态 扩展说明

python app.py 适合开发，PyInstaller + pywebview 适合交给同学或老师使用。两者复用同一套 Flask 后端和前端代码，降低维护成本。

在本项目中，部署形态并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.17 数据稀疏性 扩展说明

推荐系统最常见的问题是 user-item matrix 极其稀疏。本项目虽然只有 demo_user，但仍然通过 item features 缓解稀疏性。LightFM 不只学习某首歌的 ID，也学习 genre、artist、tag 和音频分桶 token，因此即使某首候选歌从未被用户交互过，只要它拥有相似 item features，模型仍然可以给出合理分数。

在本项目中，数据稀疏性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.18 冷启动 扩展说明

冷启动包括用户冷启动和物品冷启动。新用户没有交互时，系统使用热门歌曲或默认内容画像；新歌曲没有交互时，只要有 item features，LightFM 和 content-based baseline 都能参与排序。

在本项目中，冷启动并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.19 特征估算 扩展说明

在线 API 通常提供 metadata 而不是音频分析结果。项目使用 genre hints 和 artist/title hints 估算特征，是教学项目中的可解释折中。估算结果不能声称是真实音频分析，但可以支撑训练流程展示。

在本项目中，特征估算并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.20 可解释性 扩展说明

推荐解释不依赖语言模型，而是由 _matched_features 规则生成。这样解释可以追溯到具体字段，例如 genre、tempo 和 energy。对于课堂项目，确定性解释比生成式解释更可靠。

在本项目中，可解释性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.21 鲁棒性 扩展说明

所有网络请求都 `timeout=8` 并捕获异常，所有 JSON 返回都经过 NaN 清洗，所有 CSV 都由 `ensure_data_files` 自动创建。这些工程细节保证展示时即使网络失败也不会整站崩溃。

在本项目中，鲁棒性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 `interaction matrix`；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 `query`，处理步骤是本地搜索和在线 `provider fallback`，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 `Dataset`、`interactions`、`item features` 和 `fit`，输出是 `pkl` 模型，设计原因是 `hybrid recommendation`，失败处理是 `fallback baseline`。

18.22 模型评估 扩展说明

项目当前重展示流程，未来可以加入 `precision@k`、`recall@k`、`AUC`、`coverage`、`diversity` 等指标。LightFM 自带 `evaluation` 工具，可以用于更正式的实验报告。

在本项目中，模型评估并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 `interaction matrix`；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 `query`，处理步骤是本地搜索和在线 `provider fallback`，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 `Dataset`、`interactions`、`item features` 和 `fit`，输出是 `pkl` 模型，设计原因是 `hybrid recommendation`，失败处理是 `fallback baseline`。

18.23 版权边界 扩展说明

项目只处理 `metadata`，不下载或播放音乐。它尊重音乐平台版权限制，适合作为教育项目展示推荐系统，而不是音乐播放或下载工具。

在本项目中，版权边界并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 `interaction matrix`；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 `query`，处理步骤是本地搜索和在线 `provider fallback`，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 `songs.csv` 与 `interactions.csv`，处理步骤是 `Dataset`、`interactions`、`item features` 和 `fit`，输出是 `pkl` 模型，设计原因是 `hybrid recommendation`，失败处理是 `fallback baseline`。

18.24 部署形态 扩展说明

python app.py 适合开发，PyInstaller + pywebview 适合交给同学或老师使用。两者复用同一套 Flask 后端和前端代码，降低维护成本。

在本项目中，部署形态并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.25 数据稀疏性 扩展说明

推荐系统最常见的问题是 user-item matrix 极其稀疏。本项目虽然只有 demo_user，但仍然通过 item features 缓解稀疏性。LightFM 不只学习某首歌的 ID，也学习 genre、artist、tag 和音频分桶 token，因此即使某首候选歌从未被用户交互过，只要它拥有相似 item features，模型仍然可以给出合理分数。

在本项目中，数据稀疏性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.26 冷启动 扩展说明

冷启动包括用户冷启动和物品冷启动。新用户没有交互时，系统使用热门歌曲或默认内容画像；新歌曲没有交互时，只要有 item features，LightFM 和 content-based baseline 都能参与排序。

在本项目中，冷启动并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，

输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.27 特征估算 扩展说明

在线 API 通常提供 metadata 而不是音频分析结果。项目使用 genre hints 和 artist/title hints 估算特征，是教学项目中的可解释折中。估算结果不能声称是真实音频分析，但可以支撑训练流程展示。

在本项目中，特征估算并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.28 可解释性 扩展说明

推荐解释不依赖语言模型，而是由 _matched_features 规则生成。这样解释可以追溯到具体字段，例如 genre、tempo 和 energy。对于课堂项目，确定性解释比生成式解释更可靠。

在本项目中，可解释性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.29 鲁棒性 扩展说明

所有网络请求都 timeout=8 并捕获异常，所有 JSON 返回都经过 NaN 清洗，所有 CSV 都由 ensure_data_files 自动创建。这些工程细节保证展示时即使网络失败也不会整站崩溃。

在本项目中，鲁棒性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型

训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.30 模型评估 扩展说明

项目当前重展示流程，未来可以加入 precision@k、recall@k、AUC、coverage、diversity 等指标。LightFM 自带 evaluation 工具，可以用于更正式的实验报告。

在本项目中，模型评估并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.31 版权边界 扩展说明

项目只处理 metadata，不下载或播放音乐。它尊重音乐平台版权限制，适合作为教育项目展示推荐系统，而不是音乐播放或下载工具。

在本项目中，版权边界并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.32 部署形态 扩展说明

python app.py 适合开发，PyInstaller + pywebview 适合交给同学或老师使用。两者复用同一套 Flask 后端和前端代码，降低维护成本。

在本项目中，部署形态并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider

fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.33 数据稀疏性 扩展说明

推荐系统最常见的问题是 user-item matrix 极其稀疏。本项目虽然只有 demo_user，但仍然通过 item features 缓解稀疏性。LightFM 不只学习某首歌的 ID，也学习 genre、artist、tag 和音频分桶 token，因此即使某首候选歌从未被用户交互过，只要它拥有相似 item features，模型仍然可以给出合理分数。

在本项目中，数据稀疏性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.34 冷启动 扩展说明

冷启动包括用户冷启动和物品冷启动。新用户没有交互时，系统使用热门歌曲或默认内容画像；新歌曲没有交互时，只要有 item features，LightFM 和 content-based baseline 都能参与排序。

在本项目中，冷启动并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.35 特征估算 扩展说明

在线 API 通常提供 metadata 而不是音频分析结果。项目使用 genre hints 和 artist/title hints 估算特征，是教学项目中的可解释折中。估算结果不能声称为真实音频分析，但可以支撑训练流程展示。

在本项目中，特征估算并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.36 可解释性 扩展说明

推荐解释不依赖语言模型，而是由 _matched_features 规则生成。这样解释可以追溯到具体字段，例如 genre、tempo 和 energy。对于课堂项目，确定性解释比生成式解释更可靠。

在本项目中，可解释性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.37 鲁棒性 扩展说明

所有网络请求都 timeout=8 并捕获异常，所有 JSON 返回都经过 NaN 清洗，所有 CSV 都由 ensure_data_files 自动创建。这些工程细节保证展示时即使网络失败也不会整站崩溃。

在本项目中，鲁棒性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.38 模型评估 扩展说明

项目当前重展示流程，未来可以加入 precision@k、recall@k、AUC、coverage、diversity 等指标。LightFM 自带 evaluation 工具，可以用于更正式的实验报告。

在本项目中，模型评估并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是.pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.39 版权边界 扩展说明

项目只处理 metadata，不下载或播放音乐。它尊重音乐平台版权限制，适合作为教育项目展示推荐系统，而不是音乐播放或下载工具。

在本项目中，版权边界并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是.pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.40 部署形态 扩展说明

python app.py 适合开发，PyInstaller + pywebview 适合交给同学或老师使用。两者复用同一套 Flask 后端和前端代码，降低维护成本。

在本项目中，部署形态并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是.pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.41 数据稀疏性 扩展说明

推荐系统最常见的问题是 user-item matrix 极其稀疏。本项目虽然只有 demo_user，但仍然通过 item features 缓解稀疏性。LightFM 不只学习某首歌的 ID，也学习 genre、artist、tag 和音频分桶 token，因此即使某首候选歌从未被用户交互过，只要它拥有相似 item features，模型仍然可以给出合理分数。

在本项目中，数据稀疏性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模

型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.42 冷启动 扩展说明

冷启动包括用户冷启动和物品冷启动。新用户没有交互时，系统使用热门歌曲或默认内容画像；新歌曲没有交互时，只要有 item features，LightFM 和 content-based baseline 都能参与排序。

在本项目中，冷启动并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.43 特征估算 扩展说明

在线 API 通常提供 metadata 而不是音频分析结果。项目使用 genre hints 和 artist/title hints 估算特征，是教学项目中的可解释折中。估算结果不能声称为真实音频分析，但可以支撑训练流程展示。

在本项目中，特征估算并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是 pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.44 可解释性 扩展说明

推荐解释不依赖语言模型，而是由 _matched_features 规则生成。这样解释可以追溯到具体字段，例如 genre、tempo 和 energy。对于课堂项目，确定性解释比生成式解释更可靠。

在本项目中，可解释性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推

荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是.pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.45 鲁棒性 扩展说明

所有网络请求都 timeout=8 并捕获异常，所有 JSON 返回都经过 NaN 清洗，所有 CSV 都由 ensure_data_files 自动创建。这些工程细节保证展示时即使网络失败也不会整站崩溃。

在本项目中，鲁棒性并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是.pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

18.46 模型评估 扩展说明

项目当前重展示流程，未来可以加入 precision@k、recall@k、AUC、coverage、diversity 等指标。LightFM 自带 evaluation 工具，可以用于更正式的实验报告。

在本项目中，模型评估并不是抽象概念，而是直接体现在代码路径和数据文件中。用户每导入一首歌，系统就要决定它是否能进入训练；用户每点击一次反馈，系统就要决定如何改变 interaction matrix；每一次推荐输出，都要在模型分数、内容相似度、去重、多样性和可解释性之间取得平衡。这种从数据采集到模型训练再到前端展示的闭环，是本项目最适合 AI Fair 展示的地方。

如果在答辩中需要进一步展开，可以按照“输入是什么、处理步骤是什么、输出是什么、为什么这样设计、失败时怎么办”五个问题来讲。例如讲 API 搜索时，输入是 query，处理步骤是本地搜索和在线 provider fallback，输出是统一歌曲结构，设计原因是稳定和可训练，失败处理是返回空列表而不是抛异常。讲模型训练时，输入是 songs.csv 与 interactions.csv，处理步骤是 Dataset、interactions、item features 和 fit，输出是.pkl 模型，设计原因是 hybrid recommendation，失败处理是 fallback baseline。

19. 结论

本项目展示了一个完整、可运行、可解释、可训练的音乐推荐系统。它从歌单导入和歌曲搜索开始，把歌曲 metadata、音频特征和用户反馈转化为机器学习训练数据，训练 LightFM hybrid recommendation model，并通过模型分数与内容相似度生成推荐。系统保留了本地 Flask 网页和 macOS App 运行方式，同

时避免本地 LLM、音频下载和版权风险。对于 AI Fair 展示，最重要的讲解主线是：playlist import creates implicit feedback；Like/Dislike creates explicit feedback；each song is represented by item features；LightFM trains a hybrid model；the model predicts which songs the user is likely to enjoy；feedback improves future recommendations。